Karlsruher Institut für Technologie

Institut für Programmstrukturen und Datenorganisation

Prof. Dr.-Ing. Gregor Snelting
gregor.snelting@kit.edu

Prof. Dr. Ralf H. Reussner
reussner@kit.edu

Klausur Programmierparadigmen — Beispiellösung

SS22, 29. September 2022, 11:00 – 13:00 Uhr

Zugelassene Hilfsmittel:
Papierbasierte Quellen (Vorlesungsfolien, Übungsblätter, eigene Aufzeichnungen, Bücher, ...)
Die Verwendung von elektronischen Geräten ist verboten.

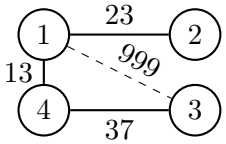
Bearbeitungszeit: 120 min

Aufgabe 1 (Haskell: Kruskal) [17 Punkte]

Im Folgenden bauen Sie eine Haskell-Version des Kruskal-Algorithmus zur Berechnung des minimalen Spannbaums nach. (Gerichtete) Graphen werden dazu als Kantenlisten dargestellt, Knoten ohne angrenzende Kanten gibt es nicht:

```
type Edge a = (Int, a, a); type Graph a = [Edge a]
```

Hier ist ein Beispiel für einen ungerichteten Graphen (alle Arten von Kanten) samt minimalem Spannbaum (nur durchgezogene Kanten) auf der linken Seite und die Kodierung in Haskell auf der rechten Seite:




```
[ (23, 1, 2), (37, 3, 4), (999, 3, 1), (13, 1, 4)
, (23, 2, 1), (37, 4, 3), (999, 1, 3), (13, 4, 1) ]
```

Wir verwenden außerdem folgende Union-Find-Datenstruktur:

```
type UF a = [(a, a)]
find :: Eq a => a -> UF a -> a
union :: a -> a -> UF a -> UF a
```

Ist `find a uf == find b uf`, so befinden sich `a` und `b` in der gleichen Zusammenhangskomponente bzgl. `uf`.

(a) Implementieren Sie die Funktion `find`, die zu einem Knoten `x` den Repräsentanten seiner Zusammenhangskomponente ermittelt. Existiert ein Paar `(x, y)` in `uf`, so gibt `find x uf` den Wert von `find y uf` zurück. Existiert kein solches Paar, gibt `find x uf` direkt `x` zurück. [5 Punkte]
Hinweis: `lookup :: Eq a => a -> [(a,b)] -> Maybe b` ist in der Prelude verfügbar.

(b) Implementieren Sie die Funktion `union`, die Zusammenhangskomponenten verschmilzt. Hierzu soll `union x y uf` das Paar der Repräsentanten von `x` und `y` zu `uf` hinzufügen. Sie sollen keine Pfadkompression implementieren. [2 Punkte]

(c) Implementieren Sie die Funktion [6 Punkte]

```
step :: Eq a => (Graph a, UF a) -> Edge a -> (Graph a, UF a)
```

die einen Schritt von Kruskals Algorithmus darstellt. Falls die Kante `e` in `step (g, uf) e` unterschiedliche Zusammenhangskomponenten verbindet (also `e = (w,a,b)` und `find a uf /= find b uf`), verschmelzen Sie beide Komponenten mit `union`, um `uf'` zu erhalten. Geben Sie dann `uf'` und den um `e` erweiterten Graphen `g` zurück. Ansonsten geben Sie `uf` und `g` unverändert zurück.

(d) Implementieren Sie die Funktion `kruskal :: Ord a => Graph a -> Graph a`, die in drei Schritten vorgeht: [4 Punkte]

1. Sortieren der Kantenliste aufsteigend nach Gewicht mithilfe von `sort :: Ord a => [a] -> [a]` aus der `Prelude`.
2. Iterieren Sie mit `step` von vorne über die sortierte Kantenliste, wobei mit dem leeren Graphen und der leeren Liste als Akkumulator `UF` gestartet wird.
3. Rückgabe des finalen, akkumulierten Graphen, der den Spannbaum darstellt.

Hinweis: Ein Fold könnte helfen.

Beispiellösung:

```
find :: Eq a => a -> UF a -> a
find a es = case lookup a es of
  Just a' -> find a' es
  Nothing -> a

union :: a -> a -> UF a -> UF a
union x y uf = (find uf x, find uf y):uf

step :: Eq a => (Graph a, UF a) -> Edge a -> (Graph a, UF a)
step (st, uf) e@(w, a, b)
  | find a uf /= find b uf = (e:st, union a b uf)
  | otherwise              = (st, uf)

kruskal :: Ord a => Graph a -> Graph a
kruskal = fst . foldl step ([], []) . sort
```

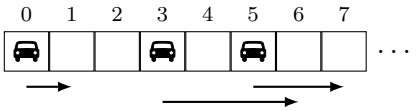
Aufgabe 2 (Haskell: Stau) [13 Punkte]

In dieser Aufgabe sollen Sie eine einfache Verkehrssimulation implementieren. Wir betrachten eine endliche Anzahl Autos, die auf einer Straße hintereinander herfahren. Die Straße ist in *Zellen* aufgeteilt, die mit ganzen Zahlen durchnummeriert sind. In einer Zelle befindet sich höchstens ein Auto. Ein Auto hat außerdem eine Geschwindkeit, die angibt, um wie viele Zellen es sich im letzten Schritt bewegt hat.

Wir modellieren also den Zustand eines Autos als Paar (Position, Geschwindigkeit) von zwei Ganzzahlen und den gesamten Zustand der Simulation als endliche Liste solcher Paare.

```
type Car = (Int, Int)
type Road = [Car]

example = [(0, 1), (3, 3), (5, 2)]
```



Zunächst sollen Sie zwei nützliche Hilfsfunktionen implementieren:

(a) Implementieren Sie die Funktion `roadAhead :: Road -> Int -> [Bool]`. [4 Punkte]
Der Aufruf `roadAhead cars p` soll die Besetzung der Zellen ab Position `p + 1` berechnen. Dabei steht `True` für eine von einem Auto in `cars` besetzte und `False` für eine freie Zelle.

Beispiel: `roadAhead example 3` ⇒ `[False, True, False, False, False, ...`

(b) Implementieren Sie die Funktion `freeCells :: [Bool] -> Int`. [2 Punkte]
`freeCells` soll die Anzahl freier Zellen am Anfang der übergebenen Liste zurückgeben.

Beispiel: `freeCells [False, False, True, False]` ⇒ `2`

Damit können wir nun die Autos simulieren.

(c) Implementieren Sie den Simulationsschritt für ein Auto in der Funktion [4 Punkte]

```
updateCar :: Road -> Car -> Int -> Car
```

Der Aufruf `updateCar road car delta` soll den neuen Zustand des Autos `car` zurückgeben. Dabei ist `road` die Liste aller Autos inklusive `car`, und `delta` gibt an, um wie viel die Geschwindigkeit des Autos in diesem Schritt erhöht oder verringert werden soll. Die neue Geschwindigkeit muss aber mindestens 0 und höchstens die Anzahl der freien Zellen vor dem Auto sein. Das Auto bewegt sich dann um diese Anzahl von Zellen weiter.

Hinweis: Das Auto mit der höchsten Position hat eine unendlich lange freie Straße vor sich. `freeCells` funktioniert also nicht ohne Weiteres.

(d) Implementieren Sie den Simulationsschritt für alle Autos in der Funktion [3 Punkte]

```
updateCars :: Road -> [Int] -> Road
```

Der Aufruf `updateCars road deltas` soll den neuen Zustand aller Autos in `road` zurückgeben. Alle Autos bewegen sich dabei gleichzeitig, d.h. sie sehen jeweils die alten Positionen der anderen Autos. Geben Sie jedem Auto einen eigenen Wert aus der Liste `deltas` als Geschwindigkeitsänderung.

Beispiellösung:

```
roadAhead :: Road -> Int -> [Bool]
roadAhead road x = map isCarAt [x + 1, x + 2 ..]
  where
    carPoss = map fst road
    isCarAt x = x `elem` carPoss

freeCells :: [Bool] -> Int
freeCells road = length (takeWhile (== False) road)

updateCar :: Road -> Car -> Int -> Car
updateCar road (x, v) delta = (x + vn, vn)
  where
    vw = max 0 (v + delta)
    vn = freeCells (take vw (roadAhead road x))

updateCars :: Road -> [Int] -> Road
updateCars road deltas =
  zipWith (\car delta -> updateCar road car delta) road deltas
```

Aufgabe 3 (Prolog: Ausführung paralleler Programme) [15 Punkte]

Ziel dieser Aufgabe ist die Simulation der Ausführung einfacher paralleler imperativer Programme. Dabei wird ein Thread-Modell verwendet. Jeder Thread ist symbolisiert als Paar `(N, Is)` aus einer Thread-Nummer `N` und einer Instruktionsliste `Is`. Ein Thread ist beendet, wenn seine Instruktionsliste leer ist. Eine Instruktion ist dabei entweder:

(a) Ein beliebiges Atom (z.B. `a`). Dies repräsentiert eine normale Operation innerhalb des Threads. Dabei steht diese nur für sich selbst, hat außer ihrem Namen also keine weiteren Eigenschaften. Eine normale Operation ist immer zulässig.

(b) `join(N)`, wobei `N` eine Thread-Nummer ist. Diese Instruktion ist genau dann zulässig, falls der Thread mit der Nummer `N` beendet ist.

Beispiel: `(1, [a, join(2), b])` ist ein Thread mit der Thread-Nummer `1` und den Instruktionen `a`, `join(2)` und `b`.

Der momentante Zustand einer Programm-Ausführung ist eine Liste von Threads (im folgenden *Thread-Pool* genannt), z.B. `[(1, [x,y]), (2, [join(1)]), (3, [])]`. In diesem Beispiel ist der Thread mit der Nummer `3` beendet.

(a) Schreiben Sie ein Prädikat `allowed(I, Ts)`. Dies soll genau dann erfüllbar sein, falls Instruktion `I` bei dem momentanen Thread-Pool `Ts` zulässig ist. [2 Punkte]
Hinweis: Das Prädikat `atom(X)` prüft, ob `X` ein Atom ist.

(b) Schreiben Sie ein Prädikat `ended(Ts)`, das genau dann erfüllt ist, wenn im Thread-Pool `Ts` alle Threads beendet sind. [2 Punkte]

Ein *Trace* beschreibt eine mögliche Ausführung eines parallelen Programms. Ein Trace ist dabei die Liste der Instruktionen in der Reihenfolge ihrer Ausführung. Das Prädikat `exec(Ts, Is)` erzeugt bei Reerfüllung alle möglichen Traces `Is` des Thread-Pools `Ts`.

Beispiel:

```
? exec([(1, [x]), (2, [y, join(1), z])], Is).
Is = [x, y, join(1), z];
Is = [y, x, join(1), z];
false.
```

Eine Race-Condition zwischen zwei Instruktionen `I1` und `I2` tritt für einen Thread-Pool auf, wenn in einer Ausführung dieses Thread-Pools `I1` vor `I2` passiert, aber in einer anderen `I2` vor `I1`.

(c) Schreiben Sie ein Prädikat `order(I1, I2, Is)`, das genau dann erfüllbar ist, falls im Trace `Is` die Instruktion `I1` vor `I2` kommt. [3 Punkte]

(d) Schreiben Sie ein Prädikat `rc(Ts, I1, I2)`, das genau dann erfüllbar ist, falls eine Race Condition zwischen den Instruktionen `I1` und `I2` für Thread-Pool `Ts` auftritt. [4 Punkte]

(e) Implementieren Sie das Prädikat `exec(Ts, Is)` wie oben definiert. Gehen Sie dabei folgendermaßen vor: [4 Punkte]
Sind alle Threads im Thread-Pool `Ts` beendet, ist die Programmausführung mit leerem Trace beendet. Ansonsten werden folgende Schritte durchgeführt:

- Ein Thread `(N, [I|Is1])` wird nichtdeterministisch aus `Ts` entfernt. Dabei sei `Ts2` der dadurch entstehende Thread-Pool.
- Es wird geprüft, ob `I` bei dem Thread-Pool `Ts` zulässig ist.
- Der Thread `(N, Is1)` wird zu `Ts2` hinzugefügt.
- Der daraus entstandene Thread-Pool wird ausgeführt. Der resultierende Trace ist dann `I` gefolgt von dem daraus entstandenen Trace.

Beispiellösung:

```
allowed(I, _) :- atom(I).
allowed(join(N), Ts) :- member((N,[]), Ts).

ended([]).
ended([(_,[]) | Ts]) :- ended(Ts).

exec(Ts, []) :- ended(Ts).
exec(Ts, [I|Is]) :- delete(Ts, (N, [I|Is1]), Ts2),
                    allowed(I, Ts),
                    exec([(N, Is1) | Ts2], Is).

order(I1, I2, Is) :- append(_, [I1|Is1], Is), member(I2, Is1).

rc(Ts, I1, I2) :- exec(Ts, Is1), order(I1, I2, Is1),
                  exec(Ts, Is2), order(I2, I1, Is2).
```

Aufgabe 4 (Primzahltest im λ-Kalkül) [15 Punkte]

In dieser Aufgabe werden wir gemeinsam einen Primzahltest für Church-Zahlen im λ-Kalkül implementieren. Hierbei dürfen alle auf den Vorlesungsfolien vordefinierten Funktionen auf Church-Zahlen und Church-Booleans verwendet werden, insbesondere sei auf `isZero` (den Test, ob eine Church-Zahl gleich 0 ist), die Funktion `succ`, die Booleschen Operationen wie `and`, `or` und `not` sowie den Rekursionsoperator `Y` verwiesen.

Zusätzlich sei die Subtraktion auf den Church-Zahlen gegeben. Das heißt, Sie dürfen eine Funktion `minus` verwenden, für die gilt

$$\texttt{minus}\ c_m\ c_n \Rightarrow^* \begin{cases} c_{m-n} & \text{falls } m \geq n \text{ und} \\ c_0 & \text{sonst.} \end{cases}$$

(a) Zunächst wollen wir den Begriff von $m \geq n$ für natürliche Zahlen m und n ins λ-Kalkül übersetzen. Schreiben Sie also einen Ausdruck `ge`, sodass gilt [3 Punkte]

$$\texttt{ge}\ c_m\ c_n \Rightarrow^* \begin{cases} c_{\text{true}} & \text{falls } m \geq n \text{ und} \\ c_{\text{false}} & \text{sonst.} \end{cases}$$

Beispiellösung: `ge = λm. λn. isZero (minus n m)`

(b) Mithilfe dieses Vergleichsoperators wollen wir nun den Modulo-Operator auf den natürlichen Zahlen definieren. Bitte definieren sie also einen Ausdruck `mod`, für den gilt [4 Punkte]

$$\texttt{mod}\ c_n\ c_k \Rightarrow^* c_{n \bmod k}.$$

Hinweis: Verwenden Sie den Rekursionsoperator!

Beispiellösung: `mod = Y (λmod. λn. λk. (ge n k) (mod (minus n k) k) n)`

(c) Nun können wir Teilbarkeit zu definieren: Definieren Sie einen Term `div`, für den gilt: [2 Punkte]

$$\texttt{div}\ c_k\ c_n \Rightarrow^* \begin{cases} c_{\text{true}} & \text{falls } k \text{ ein Teiler von } n \text{ ist und} \\ c_{\text{false}} & \text{sonst.} \end{cases}$$

Beispiellösung: `div = λk. λn. isZero (mod n k)`

(d) Schreiben Sie einen Ausdruck `noDivUpTo`, für welchen gilt: [4 Punkte]

$$\texttt{noDivUpTo n m} \Rightarrow^* \begin{cases} c_{\text{true}} & \text{falls } n \text{ keinen Teiler } k \text{ mit } 1 < k \leq m \text{ hat.} \\ c_{\text{false}} & \text{sonst.} \end{cases}$$

Beispiellösung: `noDivUpTo = Y (λf. λn. λm. or (ge 1 m) (and (not (div m n)) (f n (pred m))))`

(e) Als letzten Schritt können wir nun den Primzahltest selbst implementieren und hierbei verwenden, dass eine Zahl n genau dann prim ist, wenn $n \geq 2$ und sie keinen Teiler k mit $1 < k \leq (n-1)$ hat. Wir fordern also einen Term `isPrime` mit [2 Punkte]

$$\texttt{isPrime n} \Rightarrow^* \begin{cases} c_{\text{true}} & \text{falls } n \text{ eine Primzahl ist und} \\ c_{\text{false}} & \text{sonst.} \end{cases}$$

Beispiellösung: `isPrime = λn. and (ge n 2) (noDivUpTo n (pred n))`

Aufgabe 5 (Typinferenz, Unifikation) [15 Punkte]

Gegeben sei der folgende Term:

$$\vdash \lambda \mathtt{f}.\ \lambda \mathtt{g}.\ \lambda \mathtt{x}.\ \mathtt{f}\ (\mathtt{g}\ (\mathtt{f}\ \mathtt{x})) : \alpha_1$$

(a) Füllen Sie den fehlenden Inhalt inklusive der fehlenden Regelnamen in den Ableitungsbaum der Typinferenz ein, welcher auf der nächsten Seite abgebildet ist. [8 Punkte]

Hinweis: Sie dürfen Abkürzungen für häufig verwendete Typkontexte einführen und verwenden.

(b) Im Folgenden sei eine Constraintmenge C gegeben. Führen Sie den Algorithmus $\mathtt{unify}(C)$ zur Berechnung des zu C gehörenden allgemeinsten Unifikators $mgu(C)$ durch. [7 Punkte]

Sie können Schreibaufwand sparen, indem Sie die gedruckte Constraintmenge mithilfe von Durchstreichen, ersetzen von Typen, Modifikationen von Pfeilen, sowie Hinzufügen von neuen Gleichungen, verändern.

Tragen Sie den finalen Unifikator in die vorgegebenen grauen Felder ein.

$$\begin{array}{llllllll}
C = \{ & \alpha_1 & = & \alpha_2 & \rightarrow & \alpha_3, & & \\
 & \alpha_3 & = & \alpha_4 & \rightarrow & \alpha_5, & & \\
 & \alpha_5 & = & \alpha_6 & \rightarrow & \alpha_7, & & \\
 & \alpha_2 & = & \alpha_9 & \rightarrow & \alpha_7, & & \\
 & \alpha_4 & = & \alpha_{11} & \rightarrow & \alpha_9, & & \\
 & \alpha_2 & = & \alpha_6 & \rightarrow & \alpha_{11} & \} &
\end{array}$$

Name: Matrikelnummer:

$$
\begin{prooftree}
\AxiomC{$(\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6)(\mathtt{f}) = \alpha_2$}
\RightLabel{VAR}
\UnaryInfC{$\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6 \vdash \mathtt{f} : \alpha_8$}
\AxiomC{$(\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6)(\mathtt{g}) = \alpha_4$}
\RightLabel{VAR}
\UnaryInfC{$\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6 \vdash \mathtt{g} : \alpha_{10}$}
\AxiomC{$(\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6)(\mathtt{f}) = \alpha_2$}
\RightLabel{VAR}
\UnaryInfC{$\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6 \vdash \mathtt{f} : \alpha_{12}$}
\AxiomC{$(\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6)(\mathtt{x}) = \alpha_6$}
\RightLabel{VAR}
\UnaryInfC{$\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6 \vdash \mathtt{x} : \alpha_{13}$}
\RightLabel{APP}
\BinaryInfC{$\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6 \vdash \mathtt{f}\ \mathtt{x} : \alpha_{11}$}
\RightLabel{APP}
\BinaryInfC{$\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6 \vdash \mathtt{g}\ (\mathtt{f}\ \mathtt{x}) : \alpha_9$}
\RightLabel{APP}
\BinaryInfC{$\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4, \mathtt{x} : \alpha_6 \vdash \mathtt{f}\ (\mathtt{g}\ (\mathtt{f}\ \mathtt{x})) : \alpha_7$}
\RightLabel{ABS}
\UnaryInfC{$\mathtt{f} : \alpha_2, \mathtt{g} : \alpha_4 \vdash \lambda \mathtt{x}.\ \mathtt{f}\ (\mathtt{g}\ (\mathtt{f}\ \mathtt{x})) : \alpha_5$}
\RightLabel{ABS}
\UnaryInfC{$\mathtt{f} : \alpha_2 \vdash \lambda \mathtt{g}.\ \lambda \mathtt{x}.\ \mathtt{f}\ (\mathtt{g}\ (\mathtt{f}\ \mathtt{x})) : \alpha_3$}
\RightLabel{ABS}
\UnaryInfC{$\vdash \lambda \mathtt{f}.\ \lambda \mathtt{g}.\ \lambda \mathtt{x}.\ \mathtt{f}\ (\mathtt{g}\ (\mathtt{f}\ \mathtt{x})) : \alpha_1$}
\end{prooftree}
$$

$$
\begin{aligned}
mgu(C) = \Big[\ & \alpha_1 \Rightarrow (\alpha_6 \to \alpha_{11}) \to (\alpha_{11} \to \alpha_6) \to \alpha_6 \to \alpha_{11}\ , \\
& \alpha_2 \Rightarrow \alpha_6 \to \alpha_{11}\ , \\
& \alpha_3 \Rightarrow (\alpha_{11} \to \alpha_6) \to \alpha_6 \to \alpha_{11}\ , \\
& \alpha_4 \Rightarrow \alpha_{11} \to \alpha_6\ , \\
& \alpha_5 \Rightarrow \alpha_6 \to \alpha_{11}\ , \\
& \alpha_7 \Rightarrow \alpha_{11}\ , \\
& \alpha_9 \Rightarrow \alpha_6\ \Big]
\end{aligned}
$$

Aufgabe 6 (MPI) [14 Punkte]

Gegeben sei die untenstehende Methode `foo`, die als Parameter die zwei Arrays `first` und `second` sowie mit `x` die Anzahl der Elemente in ihnen entgegennimmt. Gehen Sie davon aus, dass MPI korrekt initialisiert und finalisiert wird. Gehen Sie weiter davon aus, dass `x` ein Vielfaches der Anzahl Prozesse ist und dass die beiden Arrays `first` und `second` korrekt initialisiert wurden.

```
1  double foo(int x, double first[x], double second[x]) {
2      int size, rank;
3      MPI_Comm_size(MPI_COMM_WORLD, &size);
4      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5
6      int workloads[size];
7      for (int i = 0; i < size; i++) {
8          workloads[i] = x / size;
9           if ( i < x % size ) workloads[i]++;
10     }
11
12     int start = 0;
13     for (int i = 0; i < rank; i++) {
14         start += workloads[i];
15     }
16     int end = start + workloads[rank];
17
18     for (int i = start; i < end; i++) {
19         first[i] = first[i] + second[i];
20     }
21
22     double result = 0.0;
23     for (int i = start; i < end; i++) {
24         result += first[i] / (double) x;
25     }
26
27     if ( rank == 0 ) {
28         for (int i = 1; i < size; i++) {
29             double local_result;
30             MPI_Status status;
31             MPI_Recv(&local_result, 1, MPI_DOUBLE, i, 77, MPI_COMM_WORLD,
                   &status );
32             result += local_result;
33         }
34     } else {
35         MPI_Send(&result, 1, MPI_DOUBLE, 0, 77, MPI_COMM_WORLD );
36     }
37
38     MPI_Bcast(&result, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
39
40     return result;
41 }
```

(a) Füllen Sie in den nachfolgenden Tabellen aus, wie die jeweils angegebenen Variablen nach Ausführung der in den Tabellen gegebenen Quelltext-Zeile belegt sind. Nehmen Sie an, dass die Arrays [6 Punkte]

```
first[] = {1, 2, 3, 4, 5, 6, 7, 8}
second[] = {10, 11, 12, 13, 14, 15, 16, 17}
```

mit `x = 8` übergeben werden und dass das Programm mit 4 Prozessen bearbeitet wird.

(a) **Variable: `workloads`**

Prozess Nr.	**Wert nach Zeile 10 (Lösung)**
0	2, 2, 2, 2
1	2, 2, 2, 2
2	2, 2, 2, 2
3	2, 2, 2, 2

(b) **Variable: `start`**

Prozess Nr.	**`Wert nach Zeile 16` (Lösung)**
0	0
1	2
2	4
3	6

(c) **Variable: `end`**

Prozess Nr.	**`Wert nach Zeile 16` (Lösung)**
0	2
1	4
2	6
3	8

(d) **Variable: `result`**

Prozess Nr.	**`Wert nach Zeile 25` (Lösung)**
0	3
1	4
2	5
3	6

Prozess Nr.	**`Wert nach Zeile 36` (Lösung)**
0	18
1	4
2	5
3	6

(e) **Variable: `result`**

Prozess Nr.	**Wert nach Zeile 38 (Lösung)**
0	18
1	18
2	18
3	18

Beispiellösung: Siehe Tabellen.

(b) Beschreiben Sie, welche Funktionalität von `foo` realisiert wird. [1 Punkt]

Beispiellösung: Es wird der Durchschnitt der Summe der beiden Arrays ermittelt.

(c) Nennen Sie eine kollektive MPI-Operation, die verwendet werden kann, um Zeile 27 bis einschließlich 38 äquivalent zu ersetzen. Geben Sie hierfür die MPI-Operation sowie deren Parametrisierung an. Gehen Sie davon aus, dass derselbe Puffer zum Senden und Empfangen von Daten verwendet werden kann. [3 Punkte]

Beispiellösung: MPI_Allreduce(&result, &result, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);

(d) Ist die Annahme, dass die Anzahl Elemente der Arrays ein Vielfaches der Anzahl Prozesse ist, notwendig für die oben gezeigte Implementierung der Funktion `foo`? Begründen Sie Ihre Antwort kurz. [2 Punkte]

Beispiellösung: Nein. Überschüssige Elemente werden in Zeile 9 gleichmäßig auf die ersten Prozesse verteilt.

(e) Würde das Programm weiterhin terminieren, wenn die `for`-Schleife in Zeile 28 bei `i = 0` beginnen würde? Begründen Sie Ihre Antwort kurz. [2 Punkte]

Beispiellösung: Nein, da Rank 0 dann versuchen würde, von sich selber eine Nachricht zu erhalten via `MPI_Recv`. Es existiert jedoch kein passender Aufruf von `MPI_Send`, sodass das Programm bei diesem Aufruf von `MPI_Recv` dauerhaft blockiert und nicht mehr terminiert.

Aufgabe 7 (Java und Design by Contract) [11 Punkte]

Gegeben ist die untenstehende Methode `parallelMean`, die als Parameter eine Liste `allValues` sowie die Anzahl Threads als `numThreads` entgegennimmt und darauf mittels Java-Futures den Durchschnittswert der Elemente der Liste berechnet. Damit der Durchschnitt korrekt berechnet werden kann, muss die Anzahl Elemente in der Liste ein Vielfaches der Anzahl Threads sein. Nehmen Sie daher an, dass jeder Thread immer gleich viele Elemente zugeteilt bekommt. Weiterhin sei folgende JavaDoc für die `subList`-Methode auf Listen gegeben:

`List<E> subList(int fromIndex, int toIndex)`: Gibt eine Ansicht des Teils der Liste zwischen dem angegebenen fromIndex (einschließlich) und toIndex (ausschließlich) zurück. Wenn fromIndex und toIndex gleich sind, ist die zurückgegebene Liste leer.
Wirft: `IndexOutOfBoundsException` - für einen ungültigen Indexwert (fromIndex < 0 || toIndex > size || fromIndex > toIndex)

```java
public double parallelMean(final List<Double> allValues, int numThreads) {
    List<Double> resultList = new ArrayList<>();
    int blockSize = allValues.size() / numThreads;

    ExecutorService pool = Executors.newFixedThreadPool(numThreads);

    for (int i = 0; i < allValues.size(); i += blockSize) {
        List<Double> subList = allValues.subList(i, i + blockSize);
        Future<Double> future = pool.submit(() -> {
            double sum = 0;
            for (double value : subList) {
                sum += value;
            }
            return sum / subList.size();
        });

        try {
            resultList.add(future.get());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    Double result = 0.0;
    for (int i = 0; i < resultList.size(); i++) {
        result += resultList.get(i);
    }
    result = result / resultList.size();
    return result;
}
```

(a) Macht die Methode `parallelMean` effektiven Nutzen von Parallelität? Begründen Sie Ihre Antwort kurz. Wenn nein, wie müssen Sie den Code modifizieren, damit es besser geht? Beschreiben Sie die gegebenenfalls nötigen Änderungen. Es ist nicht notwendig, Code zu schreiben. [4 Punkte]

Beispiellösung: Direkt nach dem Hinzufügen der Future wird auf ihm `get()` aufgerufen. Dies sorgt dafür, dass effektiv keine parallele Berechnung stattfindet, da direkt nach dem Erzeugen des Threads blockierend gewartet wird.
Bessere Lösung (Code nicht gefordert; Beschreibung reicht). Beschreibung sollte beinhalten: Speichern der Futures in einer vorher angelegten Liste o.ä., Submitten aller Tasks, danach Abfragen via get:

```
1  List<Future<Double>> futureList = new ArrayList<>();
2  for (int i = 0; i < allValues.size(); i += blockSize) {
3      List<Double> subList = allValues.subList(i, i + blockSize);
4      Future<Double> future = pool.submit(() -> {
5          double sum = 0;
6          for (double value : subList) {
7              sum += value;
8          }
9          return sum / subList.size();
10     });
11     futureList.add(future);
12 }
13
14 for (int i = 0; i < futureList.size(); i++) {
15     try {
16         resultList.add(futureList.get(i).get());
17     } catch (Exception e) { /* ... */  }
18 }
```

(b) Würde die Verwendung von `CompletableFutures` in obigem Beispiel einen Vorteil gegenüber `Futures` bieten? Begründen Sie Ihre Antwort kurz. [2 Punkte]
Hinweis: Listen in Java sind nicht thread-safe.

Beispiellösung: Nein. `CompletableFutures` bieten zusätzlich die `thenApply`-Methode an, in der Verhalten, welches nach Beendigung der Berechnung des Futures ausgeführt werden soll, definiert werden kann. In obigem Beispiel werden nach Beendigung der Berechnung jedoch nur die Ergebnisse derer in die Liste `resultList` eingefügt. Da Listen in Java nicht thread-safe sind, bietet die `thenApply`-Methode und damit die Verwendung von `CompletableFutures` keinen Mehrwert.

(c) Nennen Sie zwei Vorbedingungen und eine Nachbedingung in JML für die obenstehend gezeigte Implementierung der Methode `parallelMean`. Nehmen Sie an, dass JML mit `non_null` als default verwendet wird. Die trivialen Vor- und Nachbedingungen `true` und `false` sind keine gültigen Lösungen und werden nicht bepunktet. [3 Punkte]
Hinweis: JML bietet den `\sum`-Quantifikator, der wie folgt benutzt werden kann, um beispielsweise die Summe der Elemente eines Arrays `a` zu berechnen: `\sum int i; 0 <= i < a.length; a[i]`

Beispiellösung:

```
1  requires numThreads > 0;
2  requires allValues.size() % numThreads == 0;
```

```
3 ensures result == (sum int i; 0 <= i < allValues.size(); allValues.
    get(i)) / allValues.size();
4 ensures allValues.size() == \old(allValues.size())
```

(d) Ist die Precondition Availability Rule für die Referenzierung der beiden Parameter `allValues` und `numThreads` in einer Vorbedingung erfüllt? Begründen Sie Ihre Antwort kurz. [2 Punkte]

Beispiellösung: Ja, da sie für jeden Client selber wählbar und damit sichtbar sind.

Aufgabe 8 (Rekursiver Abstieg) [14 Punkte]

Gegeben sei die folgende Grammatik mit Startsymbol S.

$$
\begin{aligned}
S &\rightarrow X\ S \mid \varepsilon \\
X &\rightarrow \mathtt{a}\ Y\ \mathtt{a} \mid \mathtt{b} \mid \mathtt{c} \\
Y &\rightarrow Z\ S\ \mathtt{r} \\
Z &\rightarrow \mathtt{d}\ \mathtt{a} \mid \varepsilon
\end{aligned}
$$

(a) Geben Sie für die folgenden Produktionen die Indizmengen an. [3 Punkte]

Beispiellösung:

- $S \rightarrow X\ S$: $\{\mathtt{a}, \mathtt{b}, \mathtt{c}\}$
- $S \rightarrow \varepsilon$: $\{\mathtt{r}, \mathtt{EOF}\}$
- $Y \rightarrow Z\ S\ \mathtt{r}$: $\{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{d}, \mathtt{r}\}$
- $Z \rightarrow \varepsilon$: $\{\mathtt{a}, \mathtt{b}, \mathtt{c}, \mathtt{r}\}$

(b) Implementieren Sie nach dem aus der Vorlesung bekannten Schema für rekursive Abstiegsparser die Parser-Funktionen für S und Z. Brechen Sie bei Parserfehlern mit der globalen Funktion `error()` ab. [6 Punkte]

Lexer-Schnittstelle:

- Die Token sind Werte eines `enum` namens **`Token`** und heißen **`a`**, **`b`**, **`c`**, **`d`**, **`r`** und **`EOF`** (für das Ende der Eingabe). Token können bspw. mit `==` verglichen werden.
- `lexer.current` gibt nichtkonsumierenden Zugriff auf das aktuelle Token.
- `lexer.lex()` konsumiert das aktuelle Token.
- Die folgende Funktion ist verfügbar:

```
    void expect(Token tok) {
        if (lexer.current != tok)
            error();
        lexer.lex();
    }
```

Beispiellösung:

```
    void parseS() {
        switch(lexer.current) {
        case a: case b: case c:
            parseX();
            parseS();
            break;

        case r: case EOF:
            break;

        default:
            error();
        }
    }

    void parseZ() {
        switch(lexer.current) {
        case d:
```

```
82                lexer.lex();
83                expect(a);
84                break;
85
86            case a: case b: case c: case r:
87                break;
88
89            default:
90                error();
91            }
92        }
```

(c) Geben Sie die ersten 8 Grammatik-Ableitungsschritte an, die ein SLL-Parser beim Parsen des Wortes `abracadabra` durchführt. [4 Punkte]
Beispiellösung:

$$
\begin{aligned}
S &\Rightarrow XS \\
&\Rightarrow \mathtt{a}Y\mathtt{a}S \\
&\Rightarrow \mathtt{a}ZS\mathtt{ra}S \\
&\Rightarrow \mathtt{a}S\mathtt{ra}S \\
&\Rightarrow \mathtt{a}XS\mathtt{ra}S \\
&\Rightarrow \mathtt{ab}S\mathtt{ra}S \\
&\Rightarrow \mathtt{abra}S \\
&\Rightarrow \mathtt{abra}XS
\end{aligned}
$$

(d) Geben Sie ein anderes Wort ($\neq$ `abracadabra`) an, das von der Grammatik erzeugt wird und alle Terminale enthält. [1 Punkt]
Beispiellösung: `adarabc`

Aufgabe 9 (Java-Bytecode) [7 Punkte]

(a) Übersetzen Sie folgenden Java-Programmausschnitt ohne zu optimieren in Java-Bytecode: [5 Punkte]

```
1     if (a < b) {
2       t = a;
3       a = b;
4       b = t;
5     } else {
6       a = a - b;
7     }
```

Gehen Sie dabei davon aus, dass die `int`-Variablen `a`, `b` und `t` als lokale Variablen 1, 2 bzw. 3 verfügbar sind.

(b) Offensichtlich vertauscht der Code im then-Block des `if`-Statements die Inhalte der Variablen `a` und `b`. Geben Sie eine kürzere Bytecode-Sequenz an, die das auch tut. [2 Punkte]

Beispiellösung:

(a)

```
        iload 1
        iload 2
        cmp_ifge else
        iload 1
        istore 3
        iload 2
        istore 1
        iload 3
        istore 2
        goto end
else:   iload 1
        iload 2
        isub
        istore 1
end:    ...
```

(b)

```
        iload 1
        iload 2
        istore 1
        istore 2
```